

Programação para Ciência e Engenharia de Dados

Do dado bruto ao dado confiável



Prof. João Choma

Unicive

Você recebe:

`dados_vendas.csv`

Abre o arquivo.

Tem **2 milhões de linhas**.

E descobre:

- valores vazios;
- datas em três formatos;
- produtos duplicados;
- preços negativos;
- cidades escritas de formas diferentes;
- colunas que deveriam ser números armazenadas como texto.

O que você faz?

- A) Abre no Excel e começa a corrigir.
- B) Ignora e cria um gráfico.
- C) Treina uma IA.
- D) Constrói um processo reproduzível para transformar os dados.

D - D) Constrói um processo reproduzível para transformar os dados.

Mas essa resposta abre outra pergunta:

Ideia-chave

Como construímos esse processo?

Ao final, queremos sair daqui:

Ponto de partida

`dados.csv`

fonte



ingestão



validação



limpeza

Resultado esperado

transformação



integração



análise



produto de dados

Programa convencional

Entrada



Algoritmo



Saída

Sistema de dados

Dados



Qualidade?



Estrutura?



Transformação?



Integração?



Confiabilidade?



Saída

O dado é parte do problema.

Código correto + dados incorretos:

resultado incorreto

Código incorreto + dados corretos:

resultado incorreto

Código correto + dados corretos:

agora podemos conversar

Perguntas típicas:

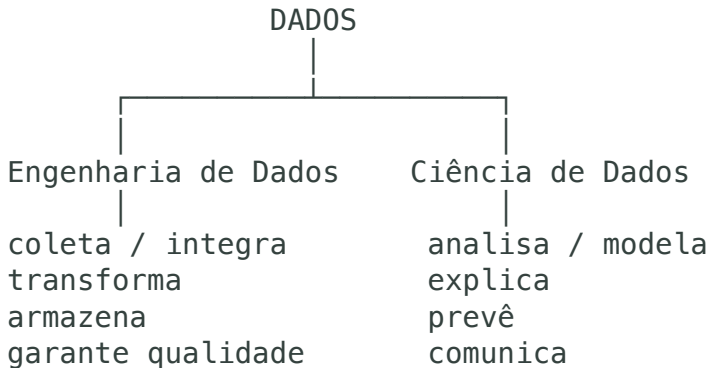
- O que aconteceu?
- Por que aconteceu?
- O que provavelmente acontecerá?
- Existe algum padrão?
- Podemos fazer uma previsão?

dados → análise → conhecimento

Perguntas típicas:

- De onde vêm os dados?
- Como chegam?
- Como são armazenados?
- Como garantir qualidade?
- Como integrar fontes?
- Como processar em escala?
- Como tornar o processo confiável?

fontes → pipelines → dados confiáveis



As fronteiras não são absolutas.

Antes de manipular dados, precisamos diferenciar três conceitos:

DADO → INFORMAÇÃO → CONHECIMENTO

Um **dado** é uma representação de um fato, evento, medida ou observação.

Exemplos:

8

95

ESC-01

01/08/2026

Isoladamente, esses valores possuem pouco contexto.

Considere:

95

O que significa?

- 95 litros?
- 95 km?
- 95 horas?
- 95 equipamentos?

O valor existe, mas ainda não sabemos interpretá-lo adequadamente.

```
equipamento = ESC-01  
data = 01/08/2026  
horas_trabalhadas = 8  
combustivel_litros = 95
```

Agora os dados descrevem um evento.

Informação é o resultado da organização, contextualização ou processamento dos dados para responder a uma pergunta.

ESC-01 | 8 horas | 95 litros

Podemos calcular:

$$95 / 8 = 11,875 \text{ L/h}$$

Informação:

Ideia-chave

O equipamento ESC-01 consumiu aproximadamente 11,88 litros por hora nessa operação.

Conhecimento surge quando informações são interpretadas em um contexto e utilizadas para compreender uma situação ou apoiar uma decisão.

Exemplo:

ESC-01 → 11,88 L/h

ESC-02 → 12,20 L/h

ESC-03 → 24,70 L/h

Podemos identificar:

Ideia-chave

O consumo do ESC-03 é muito superior ao observado nos demais.

Isso gera uma hipótese ou necessidade de investigação.

O que os dados permitem afirmar

Evidência

ESC-03 apresentou consumo por hora superior aos demais.

O que não podemos concluir automaticamente

Hipótese

ESC-03 está com defeito.

Outras explicações possíveis

- condição do terreno;
- tipo de atividade;
- carga;
- operador;
- erro de registro;
- condição mecânica.

DADO

95 litros



CONTEXTO

95 litros / ESC-01 / 8 horas



INFORMAÇÃO

11,88 litros/hora



COMPARAÇÃO + INTERPRETAÇÃO

consumo diferente do padrão



CONHECIMENTO

evidência para investigar ou decidir

DADOS BRUTOS



PREPARAÇÃO



TRANSFORMAÇÃO



ANÁLISE

ANÁLISE



INFORMAÇÃO



INTERPRETAÇÃO



CONHECIMENTO

A programação ajuda a processar e analisar os dados.

A interpretação também exige compreender o **domínio do problema**.

Dados brutos

Dados próximos da forma em que foram coletados.

Podem conter:

"ESC-01"

"esc-01"

" ESC-01 "

"ESC 01"

Dados preparados

Dados transformados segundo regras explícitas para permitir análise.

Dados não são apenas valores.

Também possuem:

- significado;
- unidade;
- contexto;
- granularidade;
- regras;
- relacionamentos.

`horas_trabalhadas = 8`

Perguntas:

- 8 horas de quê?
- De qual equipamento?
- Em qual período?
- Horas efetivas ou horímetro?
- Pode existir valor zero?
- Pode existir valor negativo?

Pergunte:

Ideia-chave

O que uma linha representa?

Essa pergunta define a **granularidade** do dataset.

equipamento		data		horas		combustível
ESC-01		01/08/2026		8		95

Vamos assumir que cada linha representa:

Ideia-chave

A operação diária de um equipamento em determinada obra.

Essa definição será importante em todas as análises.

Em praticamente tudo.

- aquisição;
- parsing;
- validação;
- limpeza;
- transformação;
- integração;
- agregação;
- análise;
- automação;
- testes;
- monitoramento.

PYTHON

Por quê?

- sintaxe acessível;
- enorme ecossistema;
- integração com bancos e APIs;
- computação científica;
- análise de dados;
- Machine Learning;
- processamento distribuído.

Python

- NumPy
- pandas
- Matplotlib
- SciPy
- scikit-learn
- PyArrow
- DuckDB
- PySpark

Precisamos dominar:

Tipos e coleções

int
float
str
bool
list
tuple
dict
set

Controle e funções

if
for
while
def
try

```
temperaturas = [23.5, 24.1, 25.8]
```

```
equipamento = {  
    "id": 17,  
    "nome": "Escavadeira 01",  
    "horas": 8421.3  
}
```

A estrutura escolhida influencia como o problema será resolvido.

Temos:

```
valores = [10, 20, 30, 40]
```

Queremos aumentar todos em 10%.

```
resultado = []  
  
for valor in valores:  
    resultado.append(valor * 1.10)
```

Funciona.

Mas...

É aqui que entra outra forma de pensar.

Vetorização

```
import numpy as np  
  
valores = np.array([10, 20, 30, 40])  
  
resultado = valores * 1.10
```

Resultado:

```
[11. 22. 33. 44.]
```

Em vez de:

Ideia-chave

Faça alguma coisa com cada elemento.

pensamos:

Ideia-chave

Faça uma operação sobre o conjunto de dados.

```
dados = np.array([  
    [10, 20, 30],  
    [40, 50, 60]  
])
```

```
dados.shape
```

```
(2, 3)
```

```
dados + 10  
dados * 2  
dados.mean()  
dados.sum()  
dados.max()
```

Código pequeno.

Operação sobre muitos dados.

```
dados = np.array([10, 50, 120, 30])
```

```
dados[dados > 40]
```

Resultado:

```
[50 120]
```

Isso será extremamente importante.

Computação numérica com arrays

NumPy — Numerical Python é uma biblioteca fundamental do ecossistema Python para computação numérica.

Ela fornece estruturas e operações eficientes para trabalhar com:

- vetores;
- matrizes;
- arrays multidimensionais;
- cálculos matemáticos;
- estatísticas;
- operações vetorizadas.

Python possui listas de propósito geral.

NumPy introduz uma estrutura especializada para computação numérica:

`numpy.ndarray`

`ndarray` significa:

N-dimensional array

```
import numpy as np
```

```
horas = np.array([  
    8, 7, 9, 6, 10  
])
```

```
print(horas)  
print(type(horas))  
print(horas.dtype)  
print(horas.shape)
```

A convenção `np` será utilizada durante a aula.

```
valores = [10, 20, 30, 40]
```

Queremos aumentar todos os valores em 10%.

```
resultado = []  
  
for valor in valores:  
    resultado.append(valor * 1.10)
```

Funciona.

Mas existe outra forma de pensar.

```
import numpy as np

valores = np.array([
    10, 20, 30, 40
])
```

Agora:

```
resultado = valores * 1.10
```

```
[11. 22. 33. 44.]
```

Observe a diferença conceitual:

LOOP

"para cada elemento faça..."

versus:

VETORIZAÇÃO

"aplique esta operação ao array"

Dados frequentemente possuem:

10 valores

1.000 valores

1.000.000 valores

Operações vetorizadas tendem a ser:

- mais expressivas;
- mais concisas;
- adequadas à computação numérica;
- frequentemente mais eficientes.

A principal estrutura do NumPy:

`np.ndarray`

Exemplo:

```
dados = np.array([
    [10, 20, 30],
    [40, 50, 60]
])
```

```
dados.shape
```

Resultado:

```
(2, 3)
```

Temos:

2 linhas

3 colunas

`dados.dtype`

O NumPy trabalha com arrays tipados.

Isso difere de uma lista Python que pode armazenar:

```
[10, "texto", True, 3.14]
```

```
dados = np.array([  
    10, 20, 30, 40, 50  
])
```

```
dados[0]
```

Resultado:

10

```
dados[1:4]
```

Resultado:

```
[20 30 40]
```

`dados + 10`

`dados * 2`

`dados / 10`

`dados ** 2`

```
dados.sum()
```

```
dados.mean()
```

```
dados.min()
```

```
dados.max()
```

```
dados = np.array([
    10, 50, 120, 30
])
```

Pergunta:

Ideia-chave

Quais valores são maiores que 40?

`dados > 40`

Resultado:

`[False True True False]`

Criamos uma **máscara booleana**.

```
dados[dados > 40]
```

Resultado:

```
[50 120]
```

Essa ideia reaparecerá no pandas.

Considere:

```
temperaturas = np.array([
    21.5, 28.2, 31.7,
    19.8, 35.1, 27.4
])
```

Encontre:

- 1 temperatura média;
- 2 maior temperatura;
- 3 menor temperatura;
- 4 temperaturas acima de 30 °C;
- 5 quantidade de temperaturas acima da média.

```
media = temperaturas.mean()

temperaturas.max()
temperaturas.min()

temperaturas[
    temperaturas > 30
]

(temperaturas > media).sum()
```

Não precisamos decorar toda a biblioteca.

Precisamos entender:

ARRAY

+

OPERAÇÕES VETORIZADAS

+

MÁSCARAS BOOLEANAS

Esses conceitos ajudam a compreender pandas.

Dados reais normalmente têm:

- linhas;
- colunas;
- tipos;
- nomes;
- valores ausentes;
- relações.

Precisamos de uma estrutura tabular.

Você recebe um arquivo:

`operacoes_equipamentos.csv`

Ele contém alguns registros.

Antes de analisar os dados:

Ideia-chave

Podemos confiar neles?

equipamento
data
horas_trabalhadas
combustivel_litros
producao_m3
operador
obra

Exemplo:

```
ESC-01,01/08/2026,8,95,120,Carlos,Obra A  
esc-01,02/08/2026,9,110,135,Ana,Obra A  
ESC 01,03/08/2026,,100,125,Carlos,Obra A  
ESC-02,03/08/2026,-4,50,80,João,Obra B
```


Os dados apresentam situações comuns do mundo real:

- valores ausentes;
- textos inconsistentes;
- tipos inadequados;
- registros duplicados;
- valores impossíveis;
- diferentes formas de representar a mesma informação.

Nosso trabalho será transformar esses dados.

Dados brutos

Dados próximos da forma em que foram coletados.

Podem conter:

"ESC-01"

"esc-01"

" ESC-01 "

"ESC 01"

Dados preparados

Dados transformados segundo regras explícitas para permitir análise.

Dados não são apenas valores.

Também possuem:

- significado;
- unidade;
- contexto;
- granularidade;
- regras;
- relacionamentos.

`horas_trabalhadas = 8`

Perguntas:

- 8 horas de quê?
- De qual equipamento?
- Em qual período?
- Horas efetivas ou horímetro?
- Pode existir valor zero?
- Pode existir valor negativo?

Pergunte:

Ideia-chave

O que uma linha representa?

Essa pergunta define a **granularidade** do dataset.

equipamento		data		horas		combustível
ESC-01		01/08/2026		8		95

Vamos assumir que cada linha representa:

Ideia-chave

A operação diária de um equipamento em determinada obra.

Essa definição será importante em todas as análises.

Um **DataFrame** é uma estrutura de dados do **pandas** utilizada para organizar informações em formato de **tabela**.

Ele possui:

- **linhas;**
- **colunas;**
- **índice;**
- **valores.**

É semelhante, visualmente, a uma **planilha** ou a uma **tabela de banco de dados**.

Podemos visualizar sua organização como:

COLUNAS			
LINHAS	equipamento	horas	combustível
	ESC-01	8	95
	ESC-02	7	87
	ESC-03	10	150

Normalmente, cada **linha** representa uma observação ou registro.

Por exemplo:

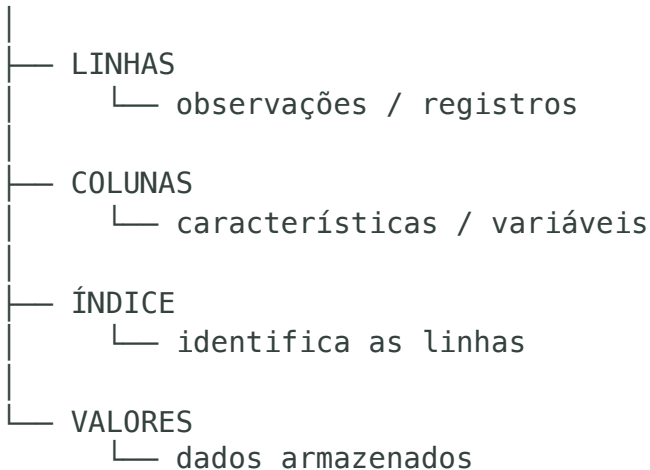
ESC-01 | 8 | 95 | 120

representa uma operação do equipamento ESC-01.

Cada **coluna** representa uma característica ou variável:

equipamento	→ código do equipamento
horas	→ horas trabalhadas
combustivel	→ combustível consumido
producao	→ produção realizada

DATAFRAME



No pandas, podemos criar um DataFrame a partir de um dicionário:

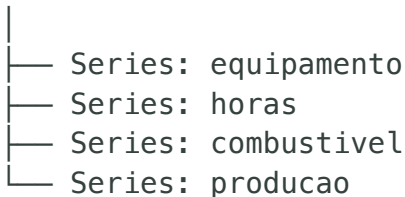
```
import pandas as pd

dados = {
    "equipamento": [
        "ESC-01", "ESC-02", "ESC-03"
    ],
    "horas": [8, 7, 10],
    "combustivel": [
        95, 87, 150
    ],
    "producao": [
        120, 100, 180
    ]
}

df = pd.DataFrame(dados)
df
```

Existe uma relação importante entre:

DataFrame



Uma forma inicial de pensar:

Ideia-chave

Um **DataFrame** é uma estrutura bidimensional organizada em linhas e colunas, na qual cada coluna pode ser manipulada como uma **Series**.

Quando utilizamos:

```
df
```

temos o DataFrame completo.

```
type(df)
```

Resultado:

```
pandas.core.frame.DataFrame
```

Mas quando selecionamos uma coluna:

```
df["horas"]
```

temos uma:

```
Series
```

Verificação do tipo

```
type(  
    df["horas"]  
)
```

Resultado:

```
pandas.core.series.Series
```

Portanto

```
df  
    ↓  
DataFrame  
  
df["horas"]  
    ↓  
Series
```

Se selecionarmos várias colunas:

```
df[  
    [  
        "equipamento",  
        "horas"  
    ]  
]
```

o resultado continua sendo um:

DataFrame

Porque podemos transformar uma tabela de dados em uma estrutura que pode ser **manipulada por código**.

Com um DataFrame podemos:

EXPLORAR



SELECIONAR



FILTRAR



LIMPAR

LIMPAR



TRANSFORMAR



AGRUPAR



ANALISAR

Primeiras linhas:

```
df.head()
```

Dimensão:

```
df.shape
```

Colunas:

```
df.columns
```

Tipos:

```
df.dtypes
```

Informações gerais:

```
df.info()
```

Uma coluna:

```
df["equipamento"]
```

Algumas colunas:

```
df[  
    [  
        "equipamento",  
        "horas"  
    ]  
]
```

Por exemplo:

Ideia-chave

Quais operações possuem mais de 8 horas?

```
df[  
    df["horas"] > 8  
]
```

Média de horas:

```
df["horas"].mean()
```

Total de combustível:

```
df["combustivel"].sum()
```

Maior produção:

```
df["producao"].max()
```

Podemos criar uma nova coluna a partir das existentes:

```
df["consumo_hora"] = (  
    df["combustivel"] /  
    df["horas"]  
)
```

Agora o DataFrame possui uma informação que não estava originalmente nos dados:

combustível + horas
↓
cálculo
↓
consumo por hora

Na prática, normalmente não criaremos todos os dados manualmente.

Eles podem estar armazenados em:

CSV
Excel
Banco de dados
API

Por exemplo:

```
df = pd.read_csv(  
    "operacoes.csv"  
)
```

O pandas transforma o arquivo

ARQUIVO CSV
↓
pd.read_csv()
↓
DATAFRAME

O DataFrame será a estrutura central da análise:

DADOS BRUTOS



CARREGAMENTO



DATAFRAME



EXPLORAÇÃO



LIMPEZA

LIMPEZA



TRANSFORMAÇÃO



ANÁLISE



INTERPRETAÇÃO

Ideia-chave

DataFrame é a principal estrutura tabular do pandas, organizada em linhas e colunas, utilizada para armazenar, manipular, transformar e analisar dados com Python.

DataFrame

=

LINHAS

+

COLUNAS

+

ÍNDICE

+

VALORES


```
import pandas as pd
```

Sua principal abstração:

DataFrame

id	equipamento	horas	consumo
1	ESC-01	8.0	95.0
2	ESC-02	7.5	87.0
3	ESC-01	9.0	110.0

pandas é uma biblioteca Python voltada à manipulação e análise de dados estruturados, especialmente dados tabulares.

Com pandas podemos:

- carregar e organizar dados;
- selecionar e filtrar registros;
- limpar e transformar valores;
- agrupar e resumir dados;
- combinar diferentes conjuntos de dados;
- produzir resultados analíticos.

NumPy



arrays + computação numérica

pandas



Series + DataFrame + dados tabulares

pandas acrescenta conceitos importantes para dados estruturados:

- índices;
- nomes de colunas;
- tipos diferentes entre colunas;
- operações orientadas a tabelas.

```
import pandas as pd  
  
print(pd.__version__)
```

Duas estruturas são fundamentais:

Series
DataFrame

```
dados = {  
    "equipamento": ["ESC-01", "ESC-02", "ESC-01"],  
    "horas": [8.0, 7.5, 9.0],  
    "consumo": [95, 87, 110]  
}
```

```
df = pd.DataFrame(dados)
```

Não comece transformando.

Comece investigando.

```
df.head()
```

```
df.shape
```

```
df.columns
```

```
df.dtypes
```

```
df.info()
```

```
df.describe()
```


- Quantas linhas existem?
- Quantas colunas?
- O que cada linha representa?
- Existe uma chave?
- Quais são os tipos?
- Existem nulos?
- Existem duplicidades?
- Qual é a unidade de medida?
- Qual é a granularidade?

Considere:

equipamento		data		consumo
ESC-01		2026-08-01		95

O que representa uma linha?

- um abastecimento?
- um dia?
- um turno?
- uma operação?
- uma máquina?

Sem saber isso, podemos calcular algo completamente errado.

```
df = pd.read_csv("operacoes.csv")
```

Parece fácil.

E é.

O problema começa depois.

```
equipamento,data,horas,combustivel  
ESC-01,01/08/2026,8,95  
esc-01,02/08/2026,9,110  
ESC 01,03/08/2026,,100  
ESC-02,2026-08-03,-4,50
```

Quantos problemas você encontrou?

```
df = pd.read_csv("operacoes.csv")
```

significa apenas:

Ideia-chave

O computador conseguiu interpretar o arquivo.

Não significa:

Ideia-chave

Os dados representam corretamente a realidade.

O trabalho começa.

Problemas comuns:

- nulos;
- duplicidades;
- tipos errados;
- unidades diferentes;
- categorias inconsistentes;
- valores impossíveis;
- erros de digitação.

```
df.isna().sum()
```

Exemplo:

equipamento	0
data	0
horas	17
combustivel	3

Depende.

Podemos:

- remover a linha;
- preencher;
- estimar;
- manter;
- sinalizar;
- recuperar na origem.


```
df = df.fillna(0)
```

Problema:

Ideia-chave

Ausente significa zero?

Talvez.

Talvez não.

```
df.duplicated().sum()
```

```
df = df.drop_duplicates()
```

Mas...

Ideia-chave

Linhas iguais são necessariamente duplicadas?

equipamento		data		litros
ESC-01		10/08/2026		100
ESC-01		10/08/2026		100

Pode ser:

- duplicação do sistema;
- dois abastecimentos legítimos.

O domínio importa.

```
df.dtypes
```

Você encontra:

```
horas    object
```

Mas deveria ser número.

```
df["horas"] = pd.to_numeric(  
    df["horas"],  
    errors="coerce"  
)
```

Valores inválidos tornam-se ausentes.

Agora podemos detectá-los.

```
df["data"] = pd.to_datetime(  
    df["data"],  
    errors="coerce"  
)
```

Datas inválidas precisam ser investigadas.

```
df["equipamento"] = (  
    df["equipamento"]  
        .str.strip()  
        .str.upper()  
)
```

Mas isso resolve:

ESC-01

ESC 01

ESC01

?

Não necessariamente.

Podemos criar uma função:

```
def normalizar Equipamento(valor):  
    valor = valor.strip().upper()  
    valor = valor.replace(" ", "-")  
    return valor
```


Agora queremos criar informação.

Dados:

horas
combustivel

Indicador:

litros por hora

```
df["litros_hora"] = (  
    df["combustivel"] / df["horas"]  
)
```

Isso é transformação de dados.

Se:

```
horas = 0
```

o que acontece?

```
df.loc[  
    df["horas"] <= 0,  
    "litros_hora"  
] = pd.NA
```

Não basta fazer a conta.

Precisamos garantir que a conta faça sentido.

```
df[df["horas"] > 8]
```

ou:

```
filtro = df["horas"] > 8
```

```
df[filtro]
```

```
df[
    (df["horas"] > 8) &
    (df["combustivel"] > 100)
]
```

```
df.sort_values(  
    "litros_hora",  
    ascending=False  
)
```

Pergunta:

Ideia-chave

Quais equipamentos apresentam maior consumo por hora?

Exemplo:

Ideia-chave

Quanto combustível cada equipamento consumiu?


```
df.groupby("equipamento") [
    "combustivel"
].sum()
```

SPLIT



APPLY



COMBINE

Agrupar → calcular → combinar.

```
df.groupby("equipamento").agg(  
    horas=("horas", "sum"),  
    combustivel=("combustivel", "sum"),  
    media_consumo=("litros_hora", "mean")  
)
```

Agora temos um produto analítico.

`equipamentos.csv`

`id | equipamento | tipo | fabricante`

Precisamos combinar os dados.

```
resultado = df.merge(  
    equipamentos,  
    on="equipamento",  
    how="left"  
)
```

OPERAÇÕES

EQUIPAMENTOS

ESC-01	—————→	ESC-01
ESC-02	—————→	ESC-02
ESC-03	—————→	???

O que fazer com ESC-03?

Se a tabela da direita tiver:

ESC-01

ESC-01

uma linha pode virar duas.

```
df.merge(  
    equipamentos,  
    on="equipamento",  
    how="left",  
    validate="many_to_one"  
)
```

Agora transformamos uma hipótese em uma verificação.

Não queremos executar comandos manualmente todo dia.

Queremos:

entrada



processo automatizado



saída

EXTRACT



TRANSFORM



LOAD

ETL

```
df = pd.read_csv("operacoes.csv")
```

```
df = limpar(df)
df = validar(df)
df = enriquecer(df)
```

```
df.to_parquet(  
    "operacoes_tratadas.parquet",  
    index=False  
)
```

Em vez de:

200 linhas no notebook...

Preferimos:

```
def carregar(): ...  
def limpar(df): ...  
def validar(df): ...  
def transformar(df): ...  
def salvar(df): ...
```

```
def pipeline():  
    df = carregar()  
    df = limpar(df)  
    validar(df)  
    df = transformar(df)  
    salvar(df)
```

Agora existe uma sequência explícita.

O programa deve verificar suas próprias suposições.

```
assert df["equipamento"].notna().all()  
assert (df["horas"] >= 0).all()  
assert df["id"].is_unique
```

Sem validação:

`dados ruins → pipeline → dados ruins mais rápido`

Com validação:

`dados ruins`



`detecção`



`falha controlada`

Imagine:

```
assert (df["preco"] >= 0).all()
```

O pipeline para.

Isso é um problema?

Não necessariamente.

Pode evitar um relatório incorreto.

- completude;
- validade;
- consistência;
- unicidade;
- acurácia;
- atualidade.

Um dataset pode estar:

100% completo

e ainda estar errado?

Sim.

CSV:

```
produto,quantidade,valor  
A,10,25.5
```

Parquet:

formato binário colunar

Vantagens:

- simples;
- legível;
- universal.

Limitações:

- tipos pouco expressivos;
- normalmente maior;
- leitura analítica menos eficiente;
- precisa interpretar texto.

Vantagens:

- colunar;
- tipos;
- compressão;
- leitura seletiva de colunas;
- excelente para analytics.

Queremos:

```
SELECT equipamento, SUM(combustivel)
FROM operacoes
GROUP BY equipamento;
```

Precisamos ler todas as colunas?

Em formatos colunares, muitas cargas analíticas podem evitar dados desnecessários.

Na verdade...

SQL está em toda parte.

Python:

```
df.groupby("equipamento")["combustivel"].sum()
```

SQL:

```
SELECT
    equipamento,
    SUM(combustivel)
FROM operacoes
GROUP BY equipamento;
```

Duas formas de expressar uma transformação.

Um banco analítico embutido permite executar SQL diretamente sobre dados locais.

Exemplo conceitual:

```
SELECT *  
FROM 'operacoes.parquet'  
WHERE horas > 8;
```

Saber alternar entre:

Python

DataFrames

SQL

conforme o problema.

Seu código funciona com:

`1.000 linhas`

E com:

`100.000.000 linhas`

`?`

Meça antes de otimizar.

Evite:

```
for _, linha in df.iterrows():  
    ...
```

quando uma operação vetorizada resolve o problema.

Menos adequado:

```
for i in range(len(df)):
    df.loc[i, "total"] = (
        df.loc[i, "qtd"] *
        df.loc[i, "valor"]
    )
```

```
df["total"] = (  
    df["qtd"] * df["valor"]  
)
```

Mais expressivo.

Geralmente mais eficiente.

Não leia o que não precisa.

```
pd.read_csv(  
    "dados.csv",  
    usecols=["data", "equipamento", "horas"]  
)
```

Se precisamos apenas de 2026:

100 milhões de registros



filtro



8 milhões de registros

As próximas operações trabalham menos.

Temos opções:

- processamento em chunks;
- consultas SQL;
- bancos analíticos;
- formatos colunares;
- processamento distribuído.

Laptop



pandas



DuckDB / engines analíticas



processamento distribuído



Spark

Não é uma regra rígida.

É uma progressão de necessidades.

Modelo distribuído para processamento de grandes volumes.

DataFrames permitem operações relacionais sobre dados distribuídos.

Big Data não significa:

Ideia-chave

Vou usar Spark porque parece profissional.

Significa:

Ideia-chave

Existe um problema de escala que justifica uma arquitetura distribuída.

Um pipeline de dados é software.

Portanto precisa de:

- legibilidade;
- modularidade;
- testes;
- versionamento;
- documentação;
- tratamento de erros;
- observabilidade.

Celula 1
Celula 7
Celula 4
Celula 12
Celula 3

Executado nessa ordem.

Funciona.

Talvez.

Outra pessoa deve conseguir:

fonte



executar processo



obter resultado

sem precisar perguntar:

Ideia-chave

"Qual célula eu rodo primeiro?"

`dados.csv`
`dados_corrigido.csv`
`dados_corrigido2.csv`
`dados_final.csv`
`dados_final_agora.csv`
`dados_final_agora_vai.csv`

```
data/raw/  
data/processed/  
src/  
tests/  
notebooks/
```

e um pipeline reproduzível.

Pergunta importante:

Ideia-chave

De onde veio este número?

Idealmente conseguimos responder:

indicador

↑

dataset tratado

↑

transformações

↑

fontes

Um pipeline executou.

Isso significa que funcionou corretamente?

Não.

- quantidade de registros;
- nulos;
- duplicidades;
- valores extremos;
- tempo de processamento;
- falhas;
- mudanças de esquema.

Ontem:

1.200.000 registros

Hoje:

12.000 registros

O pipeline terminou sem erro.

Você publica o relatório?

```
if len(df) < 100_000:  
    raise ValueError(  
        "Volume de dados inesperadamente baixo"  
    )
```

O programa incorpora conhecimento sobre o domínio.

Temos dados de equipamentos:

equipamento
data
horas
combustivel
producao
obra

Queremos descobrir eficiência operacional.

Consumo

combustível

horas

Produtividade

produção

horas

arquivos mensais



leitura



padronização



validação



limpeza

limpeza



enriquecimento



agregação



parquet



indicadores

```
import pandas as pd

df = pd.read_csv(
    "operacoes.csv"
)
```

```
print(df.shape)
print(df.dtypes)
print(df.isna().sum())
print(df.duplicated().sum())
```

```
df["equipamento"] = (  
    df["equipamento"]  
    .str.strip()  
    .str.upper()  
)
```

```
invalidos = df[
    (df["horas"] <= 0) |
    (df["combustivel"] < 0)
]
```

O que fazemos com eles?

Essa é uma decisão de domínio.

```
df["consumo_hora"] = (  
    df["combustivel"] /  
    df["horas"]  
)
```

```
resumo = (  
    df.groupby("equipamento")  
        .agg(  
            horas=("horas", "sum"),  
            combustivel=("combustivel", "sum"),  
            consumo_medio=("consumo_hora", "mean")  
        )  
)
```

```
resumo.to_parquet(  
    "resumo Equipamentos.parquet"  
)
```


Agora podemos investigar:

- qual equipamento mais trabalhou?
- qual consumiu mais?
- qual apresenta maior consumo por hora?
- existem comportamentos anormais?
- houve mudança ao longo do tempo?

Não começamos com:

Começamos com:

Dados confiáveis.

IA / ML

Analytics

Dados tratados

Pipelines confiáveis

Dados brutos

Sem base, o topo fica comprometido.

Ciência de Dados

Extrair conhecimento dos dados.

Engenharia de Dados

Construir sistemas que tornam dados disponíveis, confiáveis e utilizáveis.

Programação

O mecanismo que materializa grande parte desse processo.

Antes

Ideia-chave

Como implemento?

Ideia-chave

O que esta linha representa?

Ideia-chave

O dado é válido?

Também

Ideia-chave

Minha transformação preserva o significado?

Ideia-chave

Posso reproduzir o resultado?

Ideia-chave

Como saberei se algo der errado?

- ① Inspecione antes de transformar.
- ② Não confunda ausência com zero.
- ③ Transformações precisam preservar significado.
- ④ Valide hipóteses sobre os dados.
- ⑤ Automatize de forma reproduzível.

- ① Prefira operações expressivas e vetorizadas.
- ② SQL e Python são complementares.
- ③ Escolha formatos conforme o uso.
- ④ Escalone apenas quando necessário.
- ⑤ Pipeline de dados é software.

Python



NumPy



pandas



limpeza



transformação



agregação



integração

integração



SQL



ETL/ELT



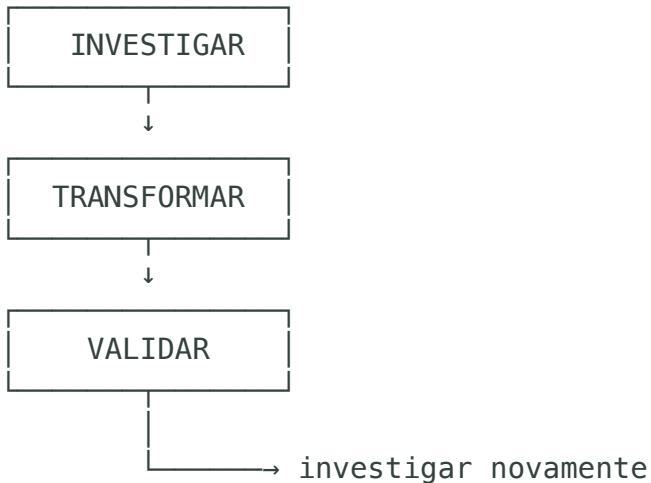
qualidade



armazenamento



escala



Você recebe amanhã:

5 GB de dados brutos

Qual é sua primeira pergunta?

Não é:

Ideia-chave

Qual algoritmo de Machine Learning vamos usar?

É algo próximo de:

Ideia-chave

O que esses dados representam e podemos confiar neles?

Agora vocês serão os engenheiros dos dados.

A atividade prática começa com dados imperfeitos.

O objetivo não será apenas "fazer funcionar".

Será construir uma solução em que possamos **confiar**.

- McKinney, W. *Python for Data Analysis*. 3rd ed.
- VanderPlas, J. *Python Data Science Handbook*.
- Kleppmann, M. *Designing Data-Intensive Applications*.
- Reis, J.; Housley, M. *Fundamentals of Data Engineering*.
- Ramalho, L. *Fluent Python*. 2nd ed.

- Python — <https://docs.python.org/>
- NumPy — <https://numpy.org/doc/>
- pandas — <https://pandas.pydata.org/docs/>
- DuckDB — <https://duckdb.org/docs/current/>
- Apache Spark — <https://spark.apache.org/docs/latest/>

Wes McKinney

Python for Data Analysis conecta Python, NumPy e pandas às tarefas de carregamento, limpeza, transformação, agregação e análise de dados.

Martin Kleppmann

Designing Data-Intensive Applications amplia a discussão para confiabilidade, escalabilidade e sistemas de dados.

Joe Reis e Matt Housley

Fundamentals of Data Engineering organiza o ciclo de vida e as responsabilidades modernas da Engenharia de Dados.

A pergunta não é apenas:

Ideia-chave

O código executou?

A pergunta é:

Ideia-chave

O resultado é confiável, explicável e reproduzível?